Software Engineering

TAHIR FAROOQ
FAST-NU

Last Lecture Review

- Black Box Testing Techniques
  - Decision Table Testing
  - Decision Table vs Test Case Table
  - Nondeterministic Table
  - State-Transition Testing

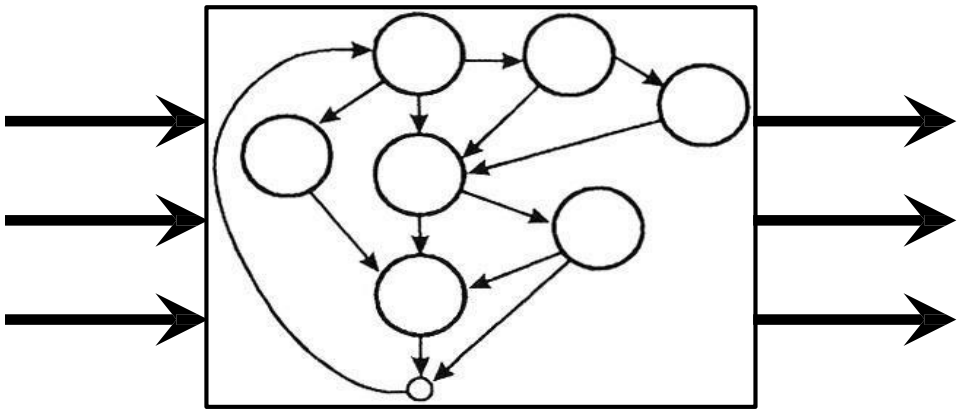
What Will You Learn Today?

- White Box Testing
- Control Flow Testing
  - Statement/Line/Basic block/Segment Coverage
  - Decision (Branch) Coverage
  - Condition Coverage
  - Multiple Condition Coverage
  - Decision/Condition Coverage
  - Loop Coverage

White Box Testing

White Box Testing

- White box testing is a strategy in which testing is based on the internal paths, structure, and implementation of the software under test (SUT)



White Box Testing - Applicability

- White box testing can be applied at all levels of system development
- Unit
- Integration
- System

- Generally white box testing is equated with unit testing performed by developers

White Box Testing

- Control flow testing
- Data flow testing

Control Flow Testing

Based on the flow of control in the program

- Logical decisions → Boolean AND, OR, XOR
- Loops
- Execution paths

Coverage metrics

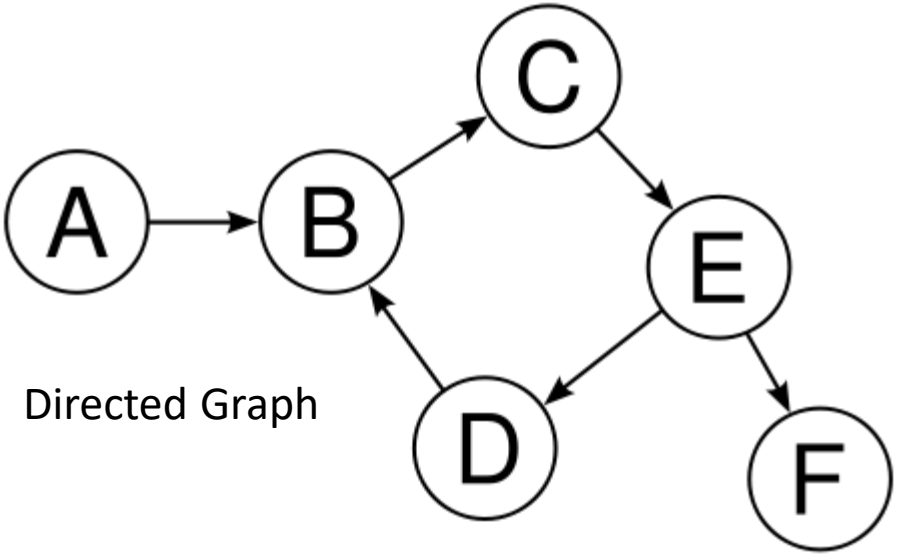
- Measure of how complete the test cases are
- Not the same as how good they are!

Which test case is good?

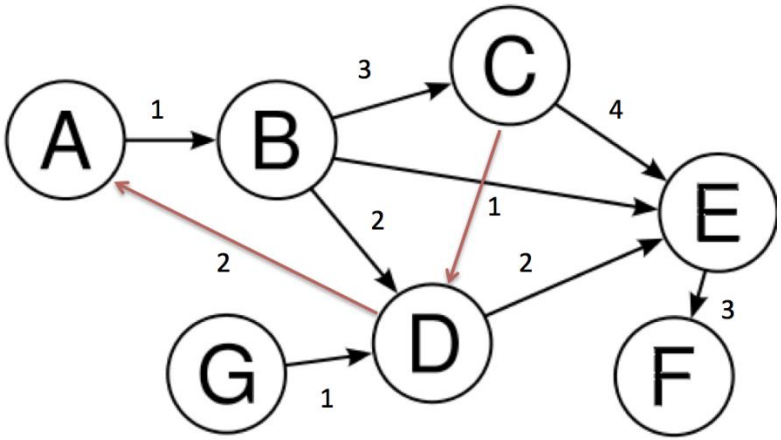
Control Flow Graphs

- Given a program written in an imperative programming language, its program graph is a directed graph in which nodes are statement fragments, and edges represent flow of control

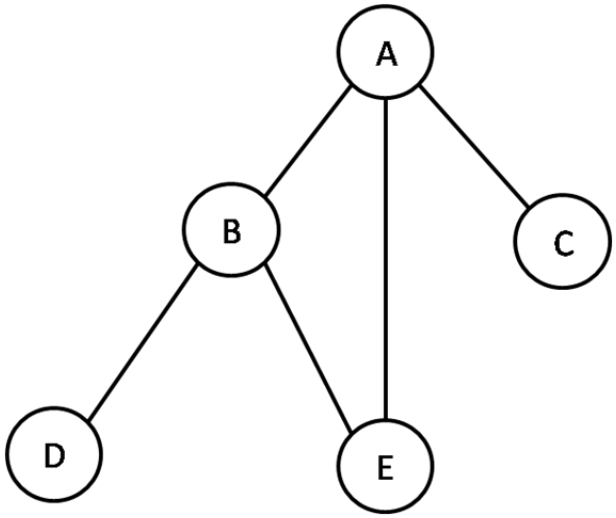
- A **basic block** is a region in the program with a single entry point and a single exit point
- This means that all jump targets start new basic blocks, and all jumps terminate basic blocks



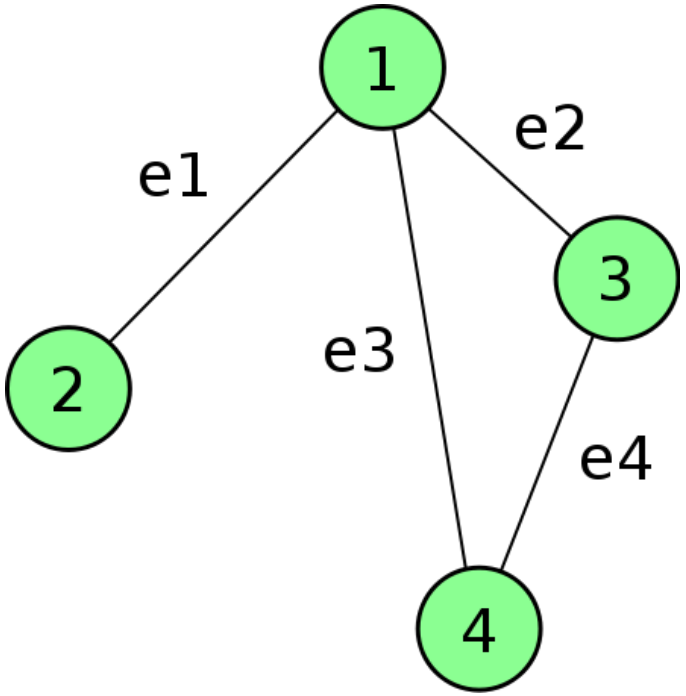
Directed Graph



Directed Weighted Graph



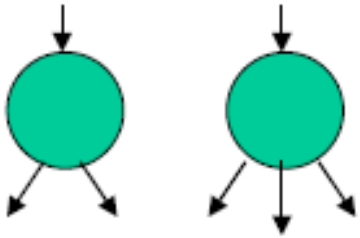
Undirected Graph



Weighted Undirected Graph

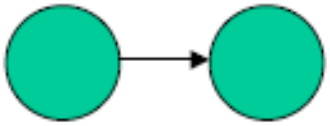
Control Flow Graphs

Process blocks

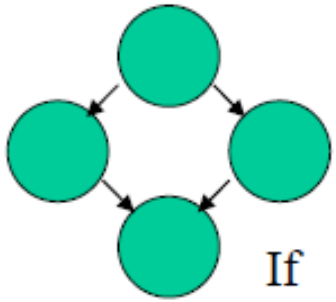


Decision Point

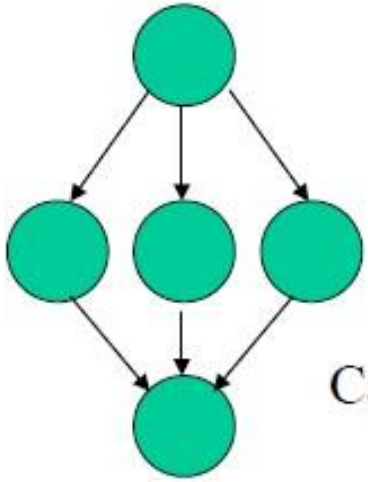
Junction Point



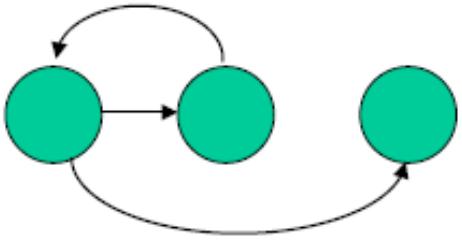
Sequence



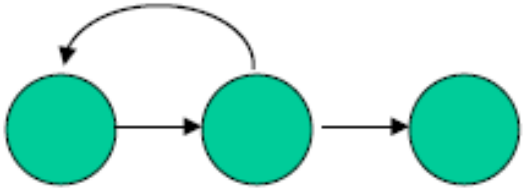
If



Case



While



Until

Code Coverage (Test Coverage Metrics)

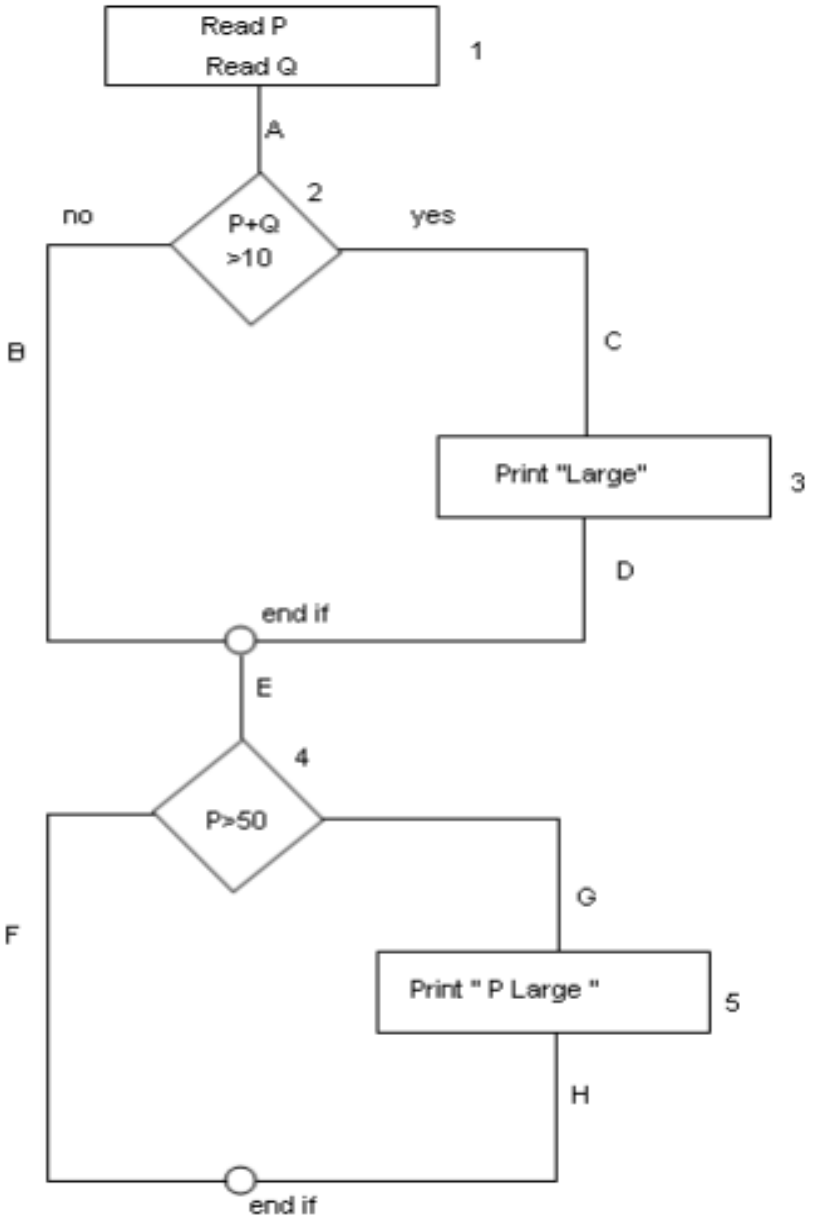
Levels of Coverage:

- Statement/Line/Basic block/Segment Coverage
- Decision (Branch) Coverage
- Condition Coverage
- Multiple Condition Coverage
- Decision/Condition Coverage
- Loop Coverage
- Path Coverage

Statement Coverage

Example:

Read P
Read Q
IF P+Q > 100 THEN
Print “Large”
ENDIF
If P > 50 THEN
Print “P Large”
ENDIF



Path Coverage (PC):

Path Coverage ensures covering of all the paths from start to end.

All possible paths are-

1A-2B-E-4F

1A-2B-E-4G-5H

1A-2C-3D-E-4G-5H

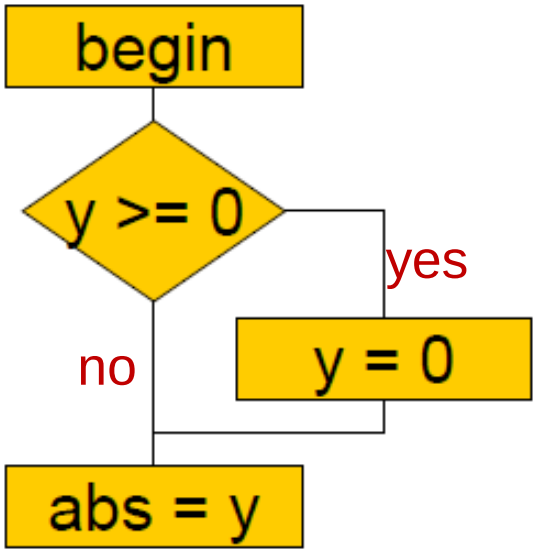
1A-2C-3D-E-4F

So path coverage is 4.

Statement Coverage

Execute each statement at least once

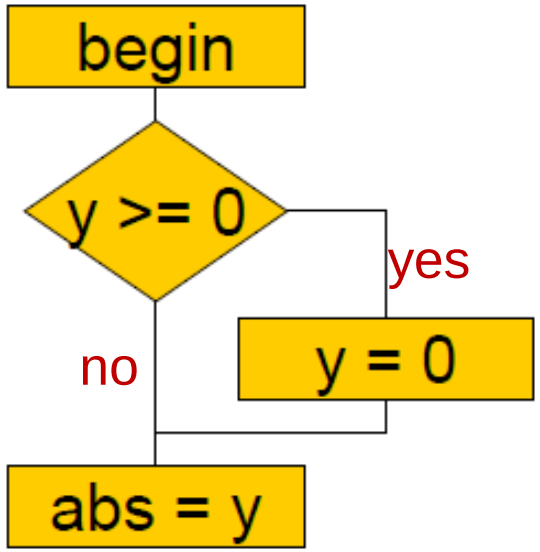
```
Begin
if ( y >= 0)
        then y = 0;
abs = y;
end;
```



Statement Coverage

Execute each statement at least once

```
Begin
if ( y >= 0)
        then y = 0;
abs = y;
end;
```



test case-1(yes):

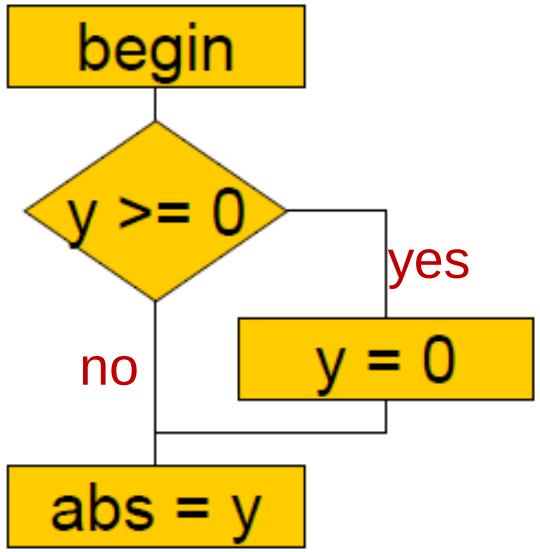
- input: y = ?
- expected result: ?
- actual result: ?

Write test cases to execute every statement!

Statement Coverage

Execute each statement at least once

```
Begin
if ( y >= 0)
        then y = 0;
abs = y;
end;
```



100% statement coverage!

test case-1(yes):

- input: y = ?
- expected result: ?
- actual result: ?

test case-1(yes):

- input: y = 0
- expected result: 0
- actual result: 0

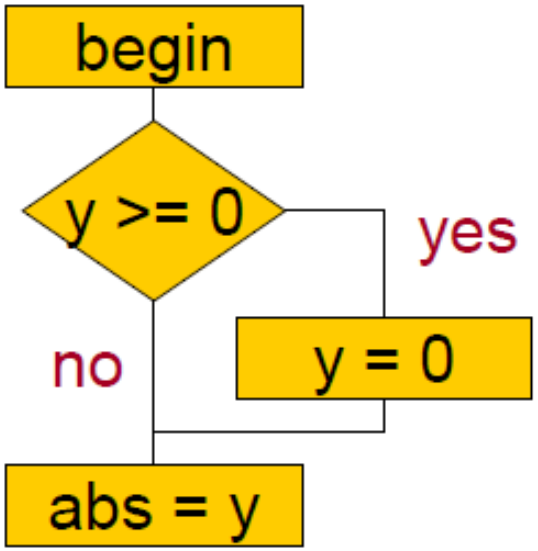
What is wrong with line coverage

- Software developers and testers commonly use line coverage because of its **simplicity** and **availability** in object code instrumentation technology.
- Of all the structural coverage criteria, line coverage is the weakest, indicating the fewest number of test cases.
- Bugs can easily occur in the cases that line coverage cannot see.
- The most significant shortcoming of line coverage is that it fails to measure whether you test simple `if` statements with a false decision outcome.
- Experts generally recommend to only use line coverage if nothing else is available.

Decision (Branch) Coverage

Execute each edge in the CFG at least once

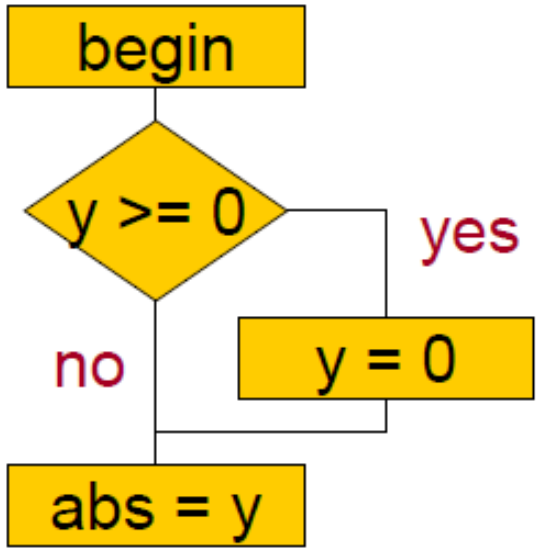
```
Begin
if ( y >= 0)
        then y = 0;
abs = y;
end;
```



Decision (Branch) Coverage

Execute each edge in the CFG at least once

```
Begin
if ( y >= 0)
        then y = 0;
abs = y;
end;
```



test case-1(yes):

- input: y = ?
- expected result: ?
- actual result: ?

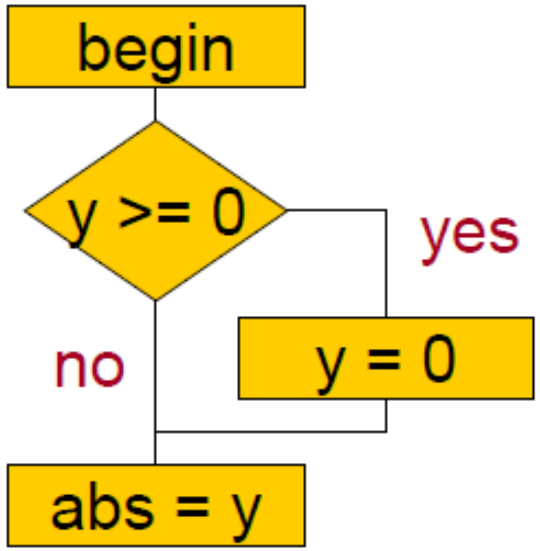
test case-2(no):

input: y = ?
expected result: ?
actual result: ?

Decision (Branch) Coverage

Execute each edge in the CFG at least once

```
Begin
if ( y >= 0)
        then y = 0;
abs = y;
end;
```



test case-1(yes):

- input: y = 0
- expected result: 0
- actual result: 0

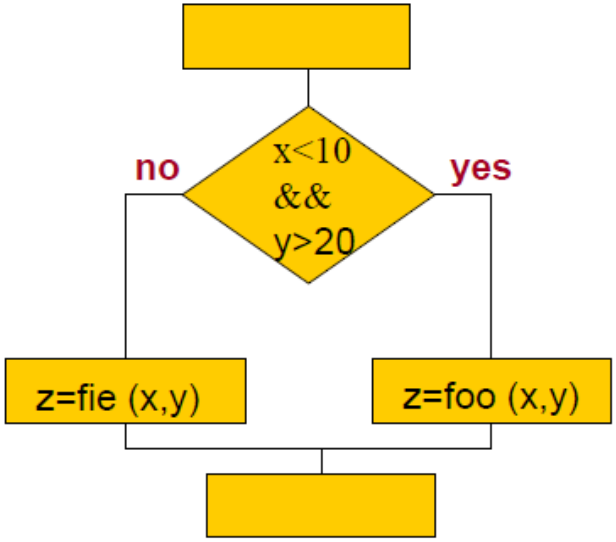
test case-2(no):

input: y = -5
expected result: 5
actual result: -5

Condition Coverage

Each condition in each decision must be both true and false

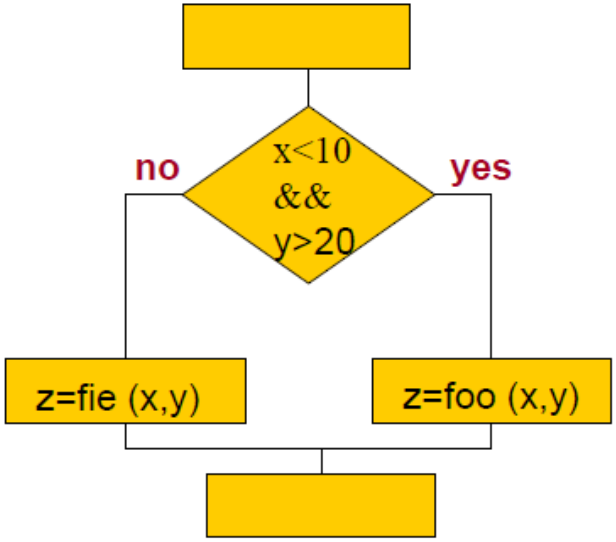
```
Begin
        if ( x < 10 && y > 20) {
                z = foo (x, y);
        else
                z =fie (x, y);
        }
end;
```



Condition Coverage

Each condition in each decision must be both true and false

```
Begin
        if ( x < 10 && y > 20) {
                z = foo (x, y);
        else
                z =fie (x, y);
        }
end;
```



test case-1(T,F):

- input: x = ?; y = ?
- expected result: ?
- actual result: ?

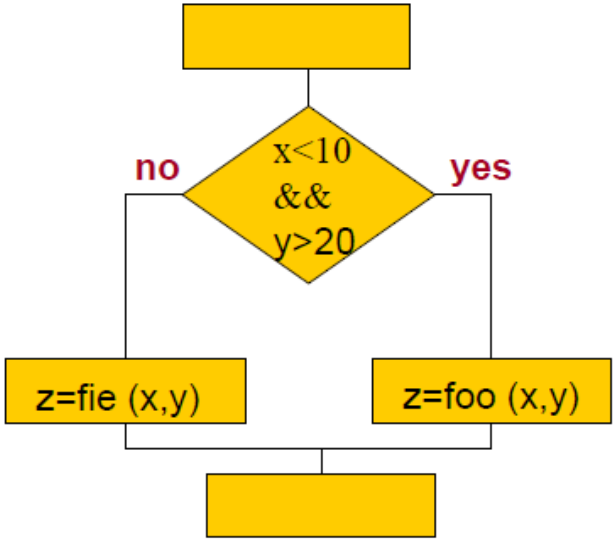
test case-2(F,T):

input: x = ?; y = ?
expected result: ?
actual result: ?

Condition Coverage

Each condition in each decision must be both true and false

```
Begin
        if ( x < 10 && y > 20) {
                z = foo (x, y);
        else
                z =fie (x, y);
        }
end;
```



test case-1(T,F):

- input: x = -4; y = 12
- expected result: ?
- actual result: ?

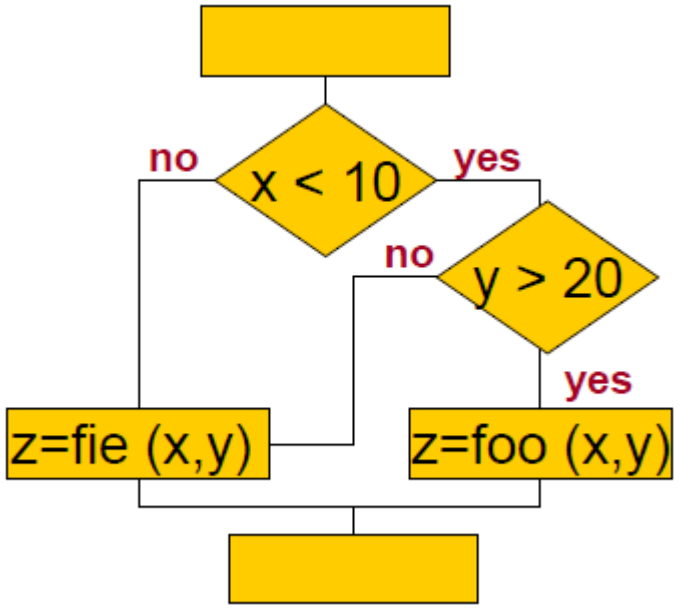
test case-2(F,T):

input: x = 12; y =30
expected result: ?
actual result: ?

Multiple Condition Coverage

All combinations of conditions must be executed in each decision

```
Begin
        if ( x < 10 && y > 20) {
                z = foo (x, y);
        else
                z =fie (x, y);
        }
end;
```

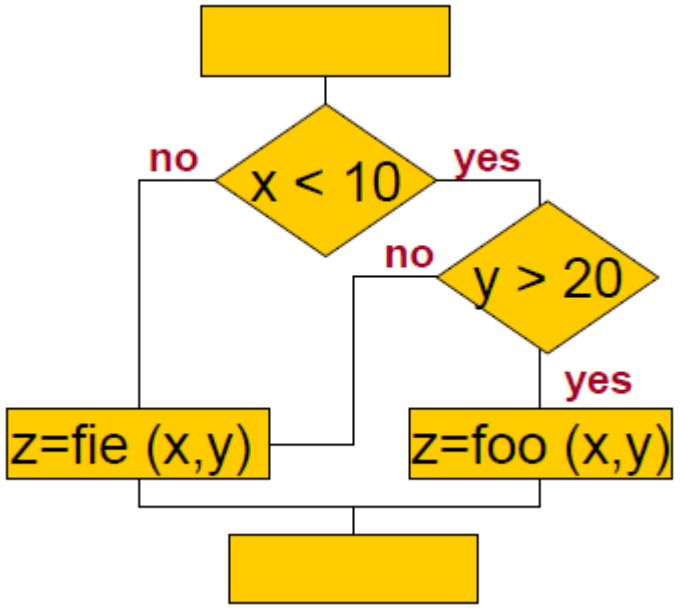


	x < ?	y > ?
▪ test-case-1:	T	T
▪ test-case:2	T	F
▪ test-case-3:	F	T
▪ test-case-4:	F	F

Multiple Condition Coverage

All combinations of conditions must be executed in each decision

```
Begin
        if ( x < 10 && y > 20) {
                z = foo (x, y);
        else
                z =fie (x, y);
        }
end;
```

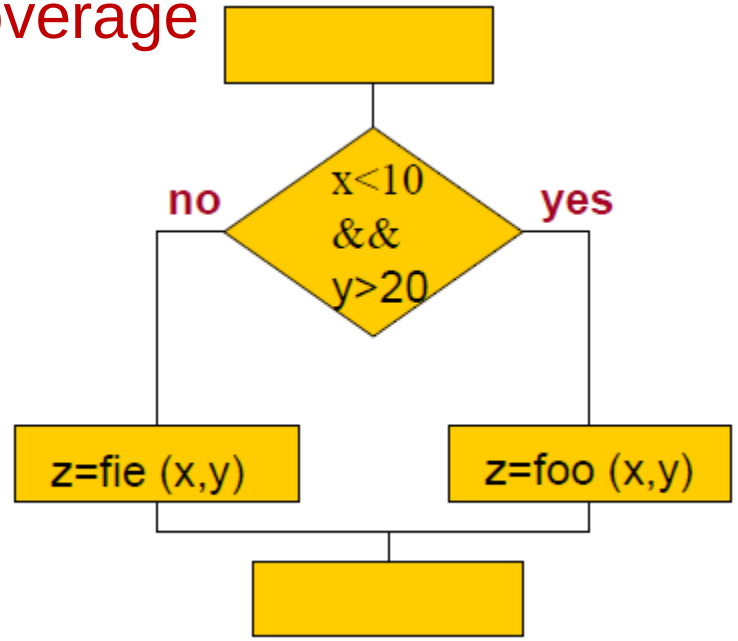


	x < 10	y > 20
test-case-1:	T	T
test-case:2	T	F
test-case-3:	F	T
test-case-4:	F	F

Decision/Condition Coverage

Satisfy both condition and decision coverage

```
Begin
        if ( x < 10 && y > 20) {
                z = foo (x, y);
        else
                z =fie (x, y);
        }
end;
```



test case-1(T, T, yes):

- input: x = ?; y = ?
- expected result: ?
- actual result: ?

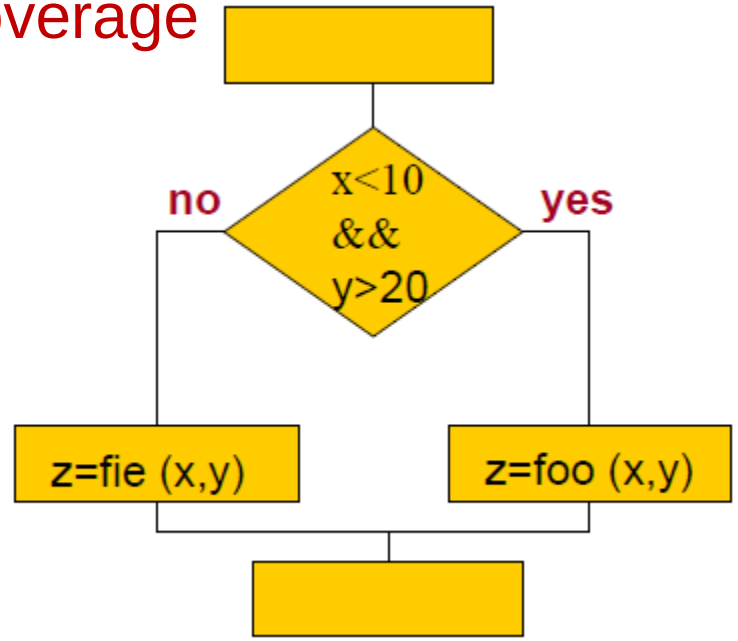
test case-2(F, F, No):

input: x = ?; y = ?
expected result: ?
actual result: ?

Decision/Condition Coverage

Satisfy both condition and decision coverage

```
Begin
        if ( x < 10 && y > 20) {
                z = foo (x, y);
        else
                z =fie (x, y);
        }
end;
```



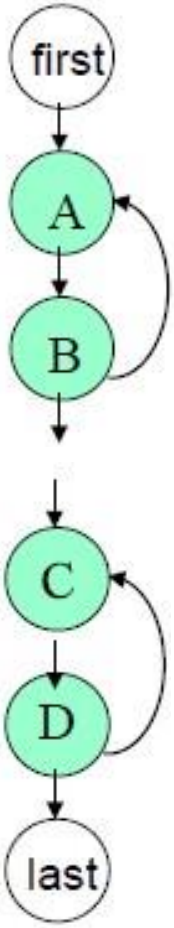
test case-1(T, T, yes):

- input: x = -4; y = 30
- expected result: ?
- actual result: ?

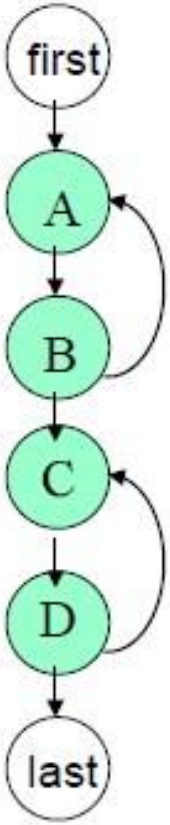
test case-2(F, F, No):

input: x = 12; y = 12
expected result: ?
actual result: ?

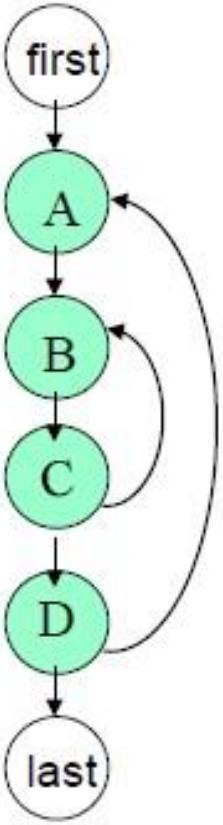
Loop Coverage



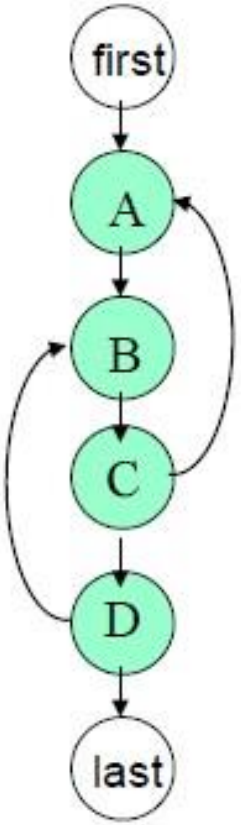
Simple



Concatenated



Nested



Unstructured

Recap

- White Box Testing
- Control Flow Testing
  - Statement/Line/Basic block/Segment Coverage
  - Decision (Branch) Coverage
  - Condition Coverage
  - Multiple Condition Coverage
  - Decision/Condition Coverage
  - Loop Coverage